Settimana 3 – Esercizi

Elementi di Informatica e Programmazione

Lab 3 – Es 1 (slide 1/2)

Progettare il programma **sortThreeStrings.py** che

- chieda all'utente di inserire tre stringhe (una per riga)
- visualizzi le stringhe in ordine lessicografico crescente (una per riga)

Il programma deve terminare con queste quattro righe di codice:

```
print("Le stringhe in ordine crescente sono:")
print(min)
print(mid)
print(max)
```

e non deve avere altri enunciati **print**

Lab 3 – Es 1 (slide 2/2)

- Ipotizzando che nella valutazione delle espressioni booleane l'interprete Python **NON** utilizzi la tecnica del "cortocircuito", calcolare il numero massimo di confronti impiegati dall'algoritmo implementato nella soluzione, nel caso peggiore. Esiste una soluzione che richiede, al massimo, **TRE soli confronti**: cercare di individuarla e realizzarla.
- Nota: "una soluzione che richiede, al massimo, tre confronti" significa "una soluzione che, nel momento in cui viene eseguita, effettua al massimo tre confronti, indipendentemente dal valore dei dati in esame". NON significa che ci debbano essere soltanto tre enunciati if... significa che devono essere eseguiti soltanto tre confronti. Ad esempio, l'esecuzione di questo frammento di codice richiede al massimo DUE confronti (soltanto uno quando x >= y).

```
if x < y:
   if z < y:
      print("pippo")
   else:
      print("pluto")
else:
   print("topolino")
```

- Per collaudare il programma, utilizzare tre stringhe molto semplici, ad esempio A, B e C. Eseguire ripetutamente il programma fornendo in ingresso tutte le (6) possibili permutazioni di tali stringhe, verificando che il programma le stampi sempre nell'ordine corretto, che è (ovviamente) A, B, C, indipendentemente dall'ordine in cui vengono fornite in input.
- **Esperimento da condurre: dove si colloca la stringa vuota nell'ordinamento lessicografico? Precede o segue qualsiasi stringa?**

Lab 3 – Es 2

Un biologo ha bisogno di un programma **Battery.py** per prevedere la crescita di una popolazione di batteri in un certo intervallo di tempo di osservazione.

Gli input sono:

- il numero iniziale di organismi
- il tasso di crescita all'ora in percentuale
- l'intervallo (in ore) di osservazione

Supponendo che nessun organismo muoia si scriva un programma che accetti questi input e visualizzi una previsione del numero totale di batteri alla fine dell'intervallo di osservazione.

Esempio di esecuzione

- numero iniziale di organismi: 5
- tasso di crescita all'ora in %: 2
- intervallo (in ore) di osservazione: 24

Previsione: il numero di batteri dopo 24.0 ore è uguale a 5

Lab 3 – Es 3a

Scrivere un programma **binarytodecimal.py** che riceva in input un numero binario, lo converta in decimale e lo stampi in output

Lab 3 – Es 3b

Scrivere un programma **decimaltobinary.py** che riceva in input un numero intero decimale, lo converta in binario e lo stampi in output

Lab 3 – Es 4 (slide 1/2)

Scrivere un programma **indovina.py** che, dati 2 numeri interi a e b (a<b) in input, generi un numero intero casuale compreso tra a e b e chieda all'utente iterativamente (finché non indovina) di indovinare il valore del numero rispondendo ai tentativi con l'indicazione "Troppo piccolo!" o "Troppo grande!"

Quando l'utente indovina il programma stampa a monitor il numero di tentativi fatti prima di indovinare

Lab 3 – Es 4 (slide 2/2)

Esempio di esecuzione:

> Scrivi il valore di a: 4

> Scrivi il valore di b: 14

> Indovina il numero: 10

«Troppo piccolo!»

> Indovina il numero: 8

«Troppo piccolo!»

> Indovina il numero: 7

«Troppo piccolo!»

> Indovina il numero: 4

«Troppo grande!»

> Indovina il numero: 5

«Hai indovinato in 5 mosse!»

Lab 3 – Es 5

Scrivete un programma **somma_e_media.py** che legga i numeri inseriti uno a uno dall'utente fino a che l'utente non inserisce la parola chiave 'fine'.

Il programma, una volta che l'utente ha inserito 'fine', stampa a monitor, la somma e la media dei numeri inseriti.

Esempio di esecuzione:

>Inserisci un numero: 4

>Inserisci un numero: 5

>Inserisci un numero: 7

>Inserisci un numero: fine

«Somma: 16»

«Media: 5.333333333333333»

Lab 3 – Es 6

Progettare il programma **someNumbers.py** che chieda all'utente di inserire una sequenza di numeri (terminata da una riga vuota) e ne visualizzi:

- la somma
- la somma dei valori assoluti
- il prodotto
- il valore medio

Il programma deve visualizzare un messaggio d'errore (e niente altro) se la sequenza di numeri termina prima che siano stati introdotti **almeno due** numeri.

Lab 3 – Es 7

Progettare il programma **standardDeviation.py** che chieda all'utente di inserire una sequenza di numeri (terminata da una riga vuota) e ne visualizzi la deviazione standard.

La deviazione standard D si calcola con la formula seguente:

```
D = Math.sqrt((A - B*B/n)/(n-1))
```

dove **n** è il numero di valori, **B** è la somma di tutti i valori e **A** è la somma dei quadrati di tutti i valori. La formula è valida soltanto per n > 1 (per n = 1 la deviazione standard vale 0, per definizione).

Il programma deve visualizzare un messaggio d'errore (e niente altro) se il flusso viene chiuso prima che siano stati introdotti **almeno due** numeri.

Lab 3 – Es 8

Scrivere il programma **euclideMCD.py** che calcoli e visualizzi il massimo comun divisore (MCD) di due numeri interi positivi m e n forniti dall'utente (con una segnalazione d'errore se l'utente fornisce numeri interi non positivi).

Si usi il ben noto ***Algoritmo di Euclide*** per il calcolo del MCD di due numeri interi positivi m ed n (con m > n):

1. Finchè il resto della divisione di m per n è diverso da zero
   - il nuovo valore di m è uguale al precedente valore di n
   - il nuovo valore di n è uguale al resto della divisione del precedente valore di m per il precedente valore di n
2. Il MCD è l'attuale valore di n

Lab 3 – Es 9 (slide 1/3)

Il gioco di Nim. Si tratta di un gioco con un certo numero di varianti: utilizzeremo la versione seguente, che ha una strategia interessante per arrivare alla vittoria. Due giocatori prelevano a turno biglie da un mucchio. In ciascun turno, il giocatore sceglie quante biglie prendere: deve prenderne almeno una, ma non oltre la metà del mucchio. Perde chi è costretto a prendere l'ultima biglia.

Scrivere il (programma-)GIOCO **nim.py** con cui un giocatore umano possa giocare a Nim contro il computer. Si genera un numero intero casuale, compreso fra 10 e 100, per indicare il numero iniziale di biglie. Si decide casualmente se la prima mossa tocca al computer o al giocatore umano. Si decide casualmente se il computer giocherà in modo intelligente o stupido. Nel modo stupido, quando è il suo turno, il computer si limita a sottrarre dal mucchio un numero casuale di biglie, purché sia una quantità ammessa. Nel modo intelligente, il computer preleva il numero di biglie sufficiente affinché il numero di quelle rimanenti sia uguale a una potenza di due diminuita di un'unità, ovvero 1, 3, 7, 15, 31 o 63: ricordate però che la mossa è valida se il giocatore non prende più della metà delle biglie del mucchio. Si può dimostrare che, se il computer preleva il numero di biglie sufficiente affinché il numero di quelle rimanenti sia uguale a una potenza di due diminuita di un'unità (biglie rimanenti = 1, 3, 7, 15, 31 o 63), si tratta sempre di una mossa valida, eccetto quando il numero delle biglie è uguale a una potenza di due diminuita di un'unità. In caso di mossa non valida il computer preleverà una quantità di biglie casuale, purché ammessa

Lab 3 – Es 9 (slide 2/3)

Si noti che, nel modo intelligente, il computer non può essere sconfitto quando ha la prima mossa, a meno che il mucchio non contenga inizialmente 15, 31 o 63 biglie. Nelle medesime condizioni, un giocatore umano che abbia la prima mossa e che conosca tale strategia vincente, può ovviamente vincere contro il computer.

Il programma deve giocare una partita dopo l'altra, fino a quando l'utente non introduce un opportuno comando che interrompe il gioco. Ogni partita è completamente indipendente dalle precedenti, sia per numero di biglie iniziali, sia per stupidità/intelligenza del computer, sia per assegnazione della prima mossa.

Lab 3 – Es 9 (slide 3/3)

Per risolvere un problema articolato come questo, occorre procedere "per gradi", individuando soluzioni di problemi intermedi che si possano collaudare, in modo da aggiungere funzionalità a uno schema già funzionante, altrimenti, se si collauda il programma soltanto alla fine, diventa ESTREMAMENTE difficile diagnosticare i problemi che si manifestano (e che ci saranno senz'altro...).

Un possibile schema da seguire potrebbe essere questo:

1. impostare un ciclo (virtualmente) infinito, il cui corpo conterrà tutto ciò che serve per giocare una partita, ma che per il momento abbia soltanto il compito di chiedere all'utente se vuole giocare un'altra partita, gestendo accuratamente tutti i casi ("Vuoi giocare ancora?": risposta S, risposta N, risposta diversa); questa fase di input richiede, a sua volta, un ciclo, che termini soltanto quando l'utente risponde S o N; risolto il problema, collaudare il programma in tutti i casi possibili e verificare che il comportamento sia sempre corretto
2. arricchire il corpo del ciclo principale (quello impostato al passo precedente), aggiungendo la fase di inizializzazione dei parametri di gioco (chi giocherà per primo? il computer sarà "intelligente" oppure no? quante biglie saranno inizialmente in gioco?), con temporanea visualizzazione di tali parametri, procedendo a un loro collaudo con più esecuzioni, verificando anche che i parametri cambino di partita in partita se il giocatore chiede di giocare di nuovo (senza eseguire di nuovo il programma)
3. concludere la soluzione del problema inserendo nel corpo del ciclo principale, tra la fase di inizializzazione dei parametri e la fase di richiesta "Vuoi giocare ancora?", un ciclo che gestisca i turni (alternando i due giocatori, umano e computer), tenendo conto della quantità di biglie rimaste nel mucchio e decidendo quando la partita è finita, decretando il vincitore; collaudare il programma completo mediante molteplici esecuzioni